

# History as Identity: The Append-Only Algebra of Spherepop

Flyxion

August 13, 2026

## Abstract

Spherepop is a computational framework whose defining departure from conventional models lies in its treatment of irreversibility as constitutive rather than incidental. Where standard semantic accounts collapse a computation to its observable output—discarding the path by which that output was reached—Spherepop preserves the ordered sequence of exclusions that produced the outcome and treats that sequence as the primary object. This essay develops the formal structure underlying this commitment alongside intuitive analogies drawn from everyday decision making, cooking, version control, and human cognition. We introduce a minimal state representation consisting of a residual option space and an accumulated commitment history, define three primitive operators—POP, BIND, and COLLAPSE—and show that the resulting algebra is essentially a free history category whose morphisms are finite sequences of irreversible commitments rather than equivalence classes of traces. The contrast with traced monoidal categories is examined in detail: where the trace operator abstracts over internal computation by collapsing feedback into a fixed-point morphism, Spherepop refuses that identification and elevates the internal path to first-class status. The identity of a Spherepop process is therefore its causal history, not its input-output relation. We argue that this makes Spherepop the natural categorical semantics for systems in which provenance, auditability, and ordered commitment matter—systems whose mathematical structure is precisely that of event-sourced append-only logs.

# Contents

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                       | <b>4</b>  |
| <b>2</b> | <b>Why Histories Matter: The Central Idea</b>             | <b>4</b>  |
| 2.1      | Programs as Functions vs. Programs as Histories . . . . . | 5         |
| 2.2      | The Restaurant Analogy . . . . .                          | 5         |
| 2.3      | The Bubble Analogy . . . . .                              | 5         |
| <b>3</b> | <b>The Trace Operator and What It Hides</b>               | <b>6</b>  |
| 3.1      | What the Trace Does . . . . .                             | 6         |
| 3.2      | What the Trace Forgets . . . . .                          | 7         |
| <b>4</b> | <b>State Representation: Option Space and History</b>     | <b>7</b>  |
| 4.1      | The Formal Definition . . . . .                           | 7         |
| 4.2      | Why Two Structures? . . . . .                             | 8         |
| 4.3      | The Asymmetry of Time . . . . .                           | 8         |
| <b>5</b> | <b>The Three Primitive Operators</b>                      | <b>8</b>  |
| 5.1      | Pop: Irreversible Commitment . . . . .                    | 9         |
| 5.2      | Bind: Contextual Restriction . . . . .                    | 9         |
| 5.3      | Collapse: Reconstruction from History . . . . .           | 10        |
| <b>6</b> | <b>A Minimal Spherepop Program</b>                        | <b>11</b> |
| <b>7</b> | <b>The History Algebra</b>                                | <b>12</b> |
| 7.1      | The Free Monoid Structure . . . . .                       | 12        |
| 7.2      | No Imposed Equivalences . . . . .                         | 12        |
| 7.3      | Concatenation as Composition . . . . .                    | 12        |
| <b>8</b> | <b>The Free History Category</b>                          | <b>13</b> |
| 8.1      | Why This Category Is “Free” . . . . .                     | 13        |
| 8.2      | Paths Rather Than Loops . . . . .                         | 14        |
| <b>9</b> | <b>Apparent Cycles from Dense Irreversible Chains</b>     | <b>14</b> |
| 9.1      | The Heartbeat Analogy . . . . .                           | 14        |
| 9.2      | Formal Statement . . . . .                                | 14        |
| 9.3      | Practical Implications . . . . .                          | 15        |

|                                                      |           |
|------------------------------------------------------|-----------|
| <b>10 Connections to Append-Only Systems</b>         | <b>15</b> |
| <b>11 Spherepop Patterns in Programming Practice</b> | <b>16</b> |
| 11.1 Scope Resolution as Spherepop . . . . .         | 16        |
| 11.2 Parser Combinators and Backtracking . . . . .   | 17        |
| <b>12 Nested Domains and Problem Solving</b>         | <b>17</b> |
| <b>13 Reactive Pipelines in Practical Activity</b>   | <b>17</b> |
| <b>14 Spherepop as a General Decision Topology</b>   | <b>18</b> |
| <b>15 Spherepop and Human Cognition</b>              | <b>19</b> |
| <b>16 Conclusion</b>                                 | <b>19</b> |

# 1 Introduction

A recurring ambition in the theory of computation is the identification of the minimal mathematical structure required to model a class of phenomena faithfully. For deterministic sequential computation that structure is the Turing machine or equivalently the  $\lambda$ -calculus; for concurrent interaction it is process algebra or the  $\pi$ -calculus; for feedback and recursion within monoidal categories it is the traced monoidal category of Joyal, Street, and Verity. In each case the theory achieves its power by deciding what to abstract over and what to preserve.

Traces, in particular, achieve their power precisely by abstraction. The trace operator  $\text{Tr}_{A,B}^U(f)$  for a morphism  $f : A \otimes U \rightarrow B \otimes U$  asserts that the precise internal behavior on the “feedback wire”  $U$  is irrelevant once its effect on the external interface  $A \rightarrow B$  is determined. The loop is collapsed; what remains is the fixed-point input-output relation.

Spherepop originates from the refusal of precisely this identification. Its central commitment is that the internal path—the ordered sequence of decisions, exclusions, and contextual restrictions that produced an observable state—is not auxiliary information to be discarded once a result is known. It is the result. Two computations that terminate at the same value but follow different causal chains are, in Spherepop’s ontology, distinct objects. The identity of a process is its history, not its output.

This essay develops the formal structure that underlies that commitment, and accompanies each stage of the formalism with intuitive analogies that make the ideas accessible before the mathematics is introduced. Section 2 presents the central idea in ordinary language. Section 3 reviews traced monoidal categories, explains the intuition behind them, and isolates precisely what the trace operator discards. Section 4 introduces the Spherepop state representation. Section 5 defines the three primitive operators and works through a minimal example. Section 7 characterizes the resulting history algebra. Section 8 gives the categorical treatment and the formal contrast with traces. Section 9 shows how apparent cyclic behavior emerges from dense chains of irreversible commitments. Sections 10 through 15 draw connections to programming, everyday problem solving, physical processes, and human reasoning. Section 16 draws the principal conclusions.

## 2 Why Histories Matter: The Central Idea

Before turning to the formal mathematics it is useful to state the central idea of Spherepop in ordinary language, and to give the reader a collection of concrete images to hold in mind while the formalism is developed.

## 2.1 Programs as Functions vs. Programs as Histories

Most computational models treat a program as a function. You provide an input, the program runs, and eventually it produces an output. Once the output is known, the intermediate steps that produced it are usually discarded. They were merely a means to an end.

Spherepop treats those intermediate steps differently. Instead of discarding them, it records them permanently. The sequence of steps becomes the primary object of study.

The same contrast appears outside computing. If you edit a document repeatedly and keep only the final version, the intermediate drafts disappear. If instead you use a version control system, every change is recorded in a commit log. The identity of the document is no longer just the final text; it is the entire sequence of edits that produced it.

Spherepop adopts the second perspective. A computation is not merely a mapping from input to output. It is a growing history of commitments, and the observable state of the system is derived from that history.

## 2.2 The Restaurant Analogy

Another illustration comes from everyday decision making. Imagine choosing a restaurant for dinner. The final choice may be “Italian.” But the path to that choice matters. Perhaps Mexican was ruled out because the restaurant was closed, and sushi was ruled out because a friend dislikes it. Two evenings may end at the same restaurant yet involve entirely different chains of reasoning.

In a purely extensional model of decision making only the final choice is recorded. In Spherepop the chain is preserved: what was eliminated, in what order, and by what constraint. Two decisions that reach the same restaurant through different chains of exclusions are therefore distinct processes, even though their observable outcome is identical.

## 2.3 The Bubble Analogy

A more visual way to picture the Spherepop state is to imagine a row of labeled bubbles floating in front of you. Each bubble represents a possible choice. As the computation proceeds, bubbles are popped one at a time. A popped bubble cannot be restored. The row of remaining bubbles shrinks, while the sequence of popped bubbles grows and is preserved as the record of what happened.

The name “Spherepop” comes from this image. The core operation of the framework is the irreversible act of popping—committing to one possibility while permanently foreclosing the others.

### 3 The Trace Operator and What It Hides

Before examining what Spherepop preserves, it is useful to be precise about what the trace operator discards.

#### 3.1 What the Trace Does

The trace can be understood informally as a formal way of saying: the internal workings of a system are irrelevant once its external behavior is known.

Suppose a program computes a function  $f : A \rightarrow B$ . The trace semantics allows the program to contain internal loops, recursive calls, and intermediate computations. However, when we describe the program mathematically, we collapse all of those internal details into the single morphism  $f$ . The program

$$\text{input} \longrightarrow \text{internal computation} \longrightarrow \text{output}$$

is identified with the mathematical function

$$A \longrightarrow B.$$

The internal sequence of operations is abstracted away.

**Film analogy.** Imagine watching a film and then writing a one-sentence plot summary. The summary may capture the overall outcome of the story, but it does not contain the sequence of scenes that produced it. Trace semantics treats the program like the summary. Spherepop treats it like the full film. Both descriptions can be accurate. The difference is entirely about what information is considered essential.

**Definition 3.1** (Traced monoidal category). *A traced symmetric monoidal category is a symmetric monoidal category  $(\mathcal{C}, \otimes, I)$  equipped with a family of functions*

$$\text{Tr}_{A,B}^U : \mathcal{C}(A \otimes U, B \otimes U) \longrightarrow \mathcal{C}(A, B),$$

*one for each triple of objects  $A, B, U$ , satisfying naturality, dinaturality, vanishing (the trace over  $I$  reduces to the identity), and the yanking equation  $\text{Tr}_{U,U}^U(\sigma_{U,U}) = \text{id}_U$ , where  $\sigma$  is the symmetry isomorphism.*

The informal reading is that a morphism  $f : A \otimes U \rightarrow B \otimes U$  can be “looped” by feeding the  $U$ -component of its output back into its  $U$ -component input, yielding a morphism  $A \rightarrow B$  that captures only the external behavior. The dinaturality condition ensures that the trace is

insensitive to the choice of representation for the internal computation: any two morphisms related by an internal change of type produce equal traces.

### 3.2 What the Trace Forgets

The trace performs a specific act of forgetting. It discards the ordered chain of intermediate steps that produced the output. Two programs that produce the same external function are identified as the same morphism regardless of their internal histories.

This identification is exactly what Spherpap refuses. Spherpap keeps the entire chain of decisions. Two computations that arrive at the same final state through different sequences of commitments remain distinct.

In categorical language, trace semantics constructs a quotient: it identifies morphisms whose internal behavior differs but whose external effect is the same. Spherpap declines that quotient and preserves the histories themselves.

**Remark 3.1.** *Consider two recursive programs that both compute the identity function  $A \rightarrow A$ . One returns immediately; the other performs a thousand recursive calls that cancel before returning. Under traced monoidal semantics both programs denote the same morphism  $\text{id}_A$ . Under Spherpap semantics the programs differ because the histories of commitments differ. The causal chains that produced the output are distinct objects.*

## 4 State Representation: Option Space and History

The primitive data structure of Spherpap is a pair representing the current state of an unfolding process.

### 4.1 The Formal Definition

**Definition 4.1** (Spherpap state). *Let  $\Omega$  be a finite set called the option space. A Spherpap state is a pair*

$$(\Omega_t, H_t)$$

*where  $\Omega_t \subseteq \Omega$  is the residual option space at time  $t$ —the set of possibilities that have not yet been excluded or committed—and  $H_t = [x_1, x_2, \dots, x_k]$  is the commitment history, an ordered list of elements that have been selected and recorded.*

The initial state of any computation is  $(\Omega, \emptyset)$ : the full option space is available and no commitments have been made. As the computation proceeds,  $\Omega_t$  shrinks monotonically

and  $H_t$  grows monotonically. These two monotonicity conditions are the central structural constraints of the framework.

## 4.2 Why Two Structures?

At first glance it might seem that only the history matters. If we know the sequence of commitments, why also track the option space?

The option space records what *could have happened but did not*. The history records what *did happen*. Both pieces of information are necessary to understand the full context of a decision.

Returning to the restaurant analogy: if the history records only the commitment [Italian], we cannot tell whether the choice was made among {Italian, Mexican, Sushi} or whether Italian was simply the only open restaurant. The option space distinguishes these scenarios by recording which alternatives were genuinely available at the time of commitment.

## 4.3 The Asymmetry of Time

There is an intentional asymmetry between the two components.

The option space  $\Omega_t$  is an unordered set. It represents the undifferentiated cloud of future possibilities, none of which has yet been actualized.

The history  $H_t$  is an ordered list. It records the temporal sequence of commitments that have actually occurred.

This asymmetry is not a notational convenience but a structural reflection of the asymmetry of time: the future is a space of undifferentiated possibilities, while the past is an ordered causal chain.

**Example 4.1.** Let  $\Omega = \{a, b, c\}$ . The initial state is  $(\{a, b, c\}, \emptyset)$ . After committing to  $a$  the state becomes  $(\{b, c\}, [a])$ . After committing to  $b$  the state becomes  $(\{c\}, [a, b])$ .

An alternative computation over the same space might produce  $(\{c\}, [b, a])$ . Both states have the same residual option space, but the histories  $[a, b]$  and  $[b, a]$  are distinct ordered lists. In *Spherepop* these are distinct processes, even if the observable state produced by COLLAPSE happens to coincide.

# 5 The Three Primitive Operators

Three operators act on *Spherepop* states. Together they constitute the complete operational vocabulary of the framework.



## 5.1 Pop: Irreversible Commitment

POP is the fundamental operator. It performs an irreversible selection from the current residual option space and records that selection in the history.

**The bubble image.** POP is the act of popping one of the floating bubbles. The bubble disappears from the row and its label is appended to the record of popped bubbles. The action cannot be undone.

**Definition 5.1 (POP).** *Let  $(\Omega_t, H_t)$  be a Spherpap state and let  $x \in \Omega_t$ . The operator  $\text{Pop}_x$  acts as*

$$\text{Pop}_x(\Omega_t, H_t) = (\Omega_t \setminus \{x\}, H_t \cdot x),$$

where  $H_t \cdot x$  denotes the list obtained by appending  $x$  to  $H_t$ .

Two changes occur simultaneously:  $x$  is removed from  $\Omega_t$ , and  $x$  is appended to  $H_t$ . The operation is irreversible because there is no operator  $\text{Pop}^{-1}$  that restores  $x$  to the option space and removes it from the history. The option space strictly shrinks with each application.

The irreversibility of POP is the mathematical expression of a structural fact about the systems Spherpap models. In any process governed by genuine commitment—the signing of a contract, the emission of a photon, the writing of a git commit, the developmental bifurcation of a cell lineage—the committed state cannot be undone. The alternatives that were not chosen at the moment of commitment are foreclosed, not merely deferred.

**Remark 5.1.** *The strict decrease in  $|\Omega_t|$  after each POP implies that a computation over a finite option space terminates after at most  $|\Omega|$  applications of POP. Spherpap is inherently finite in the basic setting; extensions to infinite or refreshed option spaces are discussed in Section 9.*

## 5.2 Bind: Contextual Restriction

BIND performs a contextual restriction of the option space without making a commitment. It applies an admissibility condition that narrows future possibilities without recording any element in the history.

**The menu analogy.** Before choosing a dish you might filter the menu by dietary restriction. This filtering removes certain options from consideration, but it is not itself a choice. BIND is the act of applying that filter. The eliminated dishes leave no trace in your record of what you ate; they were never genuine candidates given your constraints.

**Definition 5.2** (BIND). *Let  $f : 2^\Omega \rightarrow 2^\Omega$  be a function satisfying  $f(\Omega_t) \subseteq \Omega_t$  for all  $\Omega_t$ . The operator  $\text{Bind}_f$  acts as*

$$\text{Bind}_f(\Omega_t, H_t) = (f(\Omega_t), H_t).$$

The history  $H_t$  is unchanged: because no commitment has been made, there is nothing to record. The option space shrinks to  $f(\Omega_t)$ . The elements in  $\Omega_t \setminus f(\Omega_t)$  are silently excluded.

The distinction between exclusion by BIND and exclusion by POP encodes the difference between structural impossibility and chosen commitment. An element excluded by BIND was never a real candidate in the context; an element excluded by POP was a genuine alternative that the process passed over. Both kinds of elimination reduce the option space, but only the second leaves a trace in the history.

**Example 5.1.** *In  $\Omega = \{a, b, c\}$ , let  $f(\Omega_t) = \Omega_t \setminus \{c\}$ . Applying  $\text{Bind}_f$  to the initial state gives*

$$\text{Bind}_f(\{a, b, c\}, \emptyset) = (\{a, b\}, \emptyset).$$

*The element  $c$  is excluded, but the history remains empty.*

### 5.3 Collapse: Reconstruction from History

COLLAPSE derives an observable state from the accumulated commitment history.

**The ledger analogy.** A bank account does not store its balance as a fixed number. It stores a ledger of deposits and withdrawals. The current balance is computed on demand by replaying the ledger. COLLAPSE is that replay operation: starting from an initial configuration, it applies each committed transformation in historical order to reconstruct the observable state.

**Definition 5.3** (COLLAPSE). *Let  $s_0$  be a fixed root state and let  $\{T_x\}_{x \in \Omega}$  be a family of state transformations. For a history  $H_t = [x_1, x_2, \dots, x_n]$ , the operator Collapse acts as*

$$\text{Collapse}(H_t) = T_{x_n} \circ T_{x_{n-1}} \circ \dots \circ T_{x_1}(s_0).$$

The observable state is derived by composing the transformations associated with each committed element in temporal order. The state is not stored; it is always reconstructed from the history. This inversion of the usual relationship between state and history is the deepest structural claim of the Spherepop ontology.

In general  $T_a \circ T_b \neq T_b \circ T_a$ , so the histories  $[a, b]$  and  $[b, a]$  may yield different observable states via COLLAPSE. This non-commutativity is why the history must be an ordered list rather than a set.

## 6 A Minimal Spherepop Program

We now trace through the smallest nontrivial Spherepop computation in full detail.

Let  $\Omega = \{a, b, c\}$ , let the initial state be  $(\{a, b, c\}, \emptyset)$ , and let  $s_0$  be the root state.

**Step 0: Initial configuration.**

$$(\{a, b, c\}, \emptyset).$$

All three possibilities are available. No commitments have been made.

**Step 1: Apply  $\text{Bind}_f$  with  $f(\Omega_t) = \Omega_t \setminus \{c\}$ .** The constraint eliminates  $c$  from the option space:

$$(\{a, b\}, \emptyset).$$

The element  $c$  is gone, but the history is still empty. No commitment has occurred;  $c$  was ruled out by context.

**Step 2: Apply  $\text{Pop}_a$ .** The system commits to  $a$ :

$$(\{b\}, [a]).$$

The option space contains only  $b$ . The history records that  $a$  was the first commitment.

**Step 3: Apply  $\text{Collapse}([a])$ .** The observable state is reconstructed:

$$s = T_a(s_0).$$

**What this example reveals.** A traditional functional model would return  $T_a(s_0)$  and discard the intermediate structure. Spherepop preserves the pair  $(\{b\}, [a])$ . The residual  $b$  records that a genuine alternative was available at the moment of commitment. The history  $[a]$  records which alternative was chosen and in what order. The identity of the computation is this pair, not merely its output.

**Causal provenance.** Two Spherepop computations that both produce  $s = T_a(s_0)$  may differ in provenance. A computation that applies  $\text{Bind}_f$  before  $\text{Pop}_a$  leaves the state  $(\{b\}, [a])$ , indicating that  $c$  was already excluded when the commitment was made. A computation that applies only  $\text{Pop}_a$  with no prior  $\text{BIND}$  leaves  $(\{b, c\}, [a])$ , indicating that both  $b$  and  $c$  were live alternatives at the moment of commitment. These computations have the same output but different causal histories, and in Spherepop they are genuinely different objects.

## 7 The History Algebra

The operators of the previous section generate a mathematical structure we call the *history algebra*. The key observation is that commitment histories behave exactly like words in a free monoid.

### 7.1 The Free Monoid Structure

The collection of all finite ordered sequences over the option space  $\Omega$  forms the free monoid  $\Omega^*$  under concatenation. The empty sequence  $\emptyset$  is the identity element. Concatenation is associative but not commutative:  $[a, b] \neq [b, a]$  in general.

The commitment histories of SpheroPOP are precisely the elements of  $\Omega^*$ . When a new commitment occurs, the history grows by appending the new element:

$$H \cdot x = [x_1, \dots, x_n, x].$$

There are no further algebraic relations. Distinct sequences remain distinct; the free monoid imposes no equations identifying different words.

### 7.2 No Imposed Equivalences

This absence of relations is a design principle rather than an omission. By refraining from quotienting histories that produce the same observable state, the history algebra preserves the full causal provenance of each computation. The ordered sequence of commitments remains visible in the algebraic structure, ensuring that the identity of a process is determined by its history rather than by its extensional outcome.

**Definition 7.1** (History algebra). *A history algebra over a finite option space  $\Omega$  consists of: the set of all SpheroPOP states  $\{(\Omega_t, H_t) \mid \Omega_t \subseteq \Omega, H_t \in \Omega^*\}$ ; for each  $x \in \Omega_t$  the operator  $\text{Pop}_x$ ; for each admissibility function  $f$  the operator  $\text{Bind}_f$ ; and a root state  $s_0$  together with a family  $\{T_x\}_{x \in \Omega}$  defining COLLAPSE.*

### 7.3 Concatenation as Composition

The composition law of the history algebra is concatenation. If one sequence of operations extends the history by  $\tau_1$  and another extends it by  $\tau_2$ , the composite extends it by  $\tau_1 \cdot \tau_2$ . This concatenation law is associative and has the empty history as its identity—exactly the laws of a monoid.

**Git analogy.** In a git repository, applying commit  $A$  followed by commit  $B$  is different from applying commit  $B$  followed by commit  $A$  whenever the two commits modify overlapping files. The non-commutativity of the history algebra is the same non-commutativity that makes merge order matter in version control.

## 8 The Free History Category

The history algebra admits a natural categorical interpretation. Histories can be viewed as morphisms in a category whose objects represent Spherepop states.

**Definition 8.1** (Free history category). *The free history category  $\mathcal{H}(\Omega)$  over a finite option space  $\Omega$  is the category whose objects are Spherepop states  $(\Omega_t, H)$  and whose morphisms from  $(\Omega_t, H)$  to  $(\Omega'_t, H')$  are pairs  $(\sigma, \tau)$  where  $\sigma : \Omega_t \rightarrow \Omega'_t$  is a set map and  $\tau \in \Omega^*$  satisfies  $H' = H \cdot \tau$ . Composition is given by  $(\sigma', \tau') \circ (\sigma, \tau) = (\sigma' \circ \sigma, \tau \cdot \tau')$ , and identity morphisms are  $(\text{id}_{\Omega_t}, \emptyset)$ .*

**Proposition 8.1.** *The category  $\mathcal{H}(\Omega)$  is well-defined. The POP and BIND operators correspond to generating morphisms in  $\mathcal{H}(\Omega)$ , and Collapse is a functor from  $\mathcal{H}(\Omega)$  to the category of observable states.*

*Proof.* Associativity of composition follows from associativity of concatenation and of function composition. Identity morphisms satisfy the required equations because appending the empty sequence leaves any history unchanged. The functor Collapse sends a history  $[x_1, \dots, x_n]$  to  $T_{x_n} \circ \dots \circ T_{x_1}$ ; functoriality follows because concatenation of histories corresponds to composition of transformations.  $\square$

### 8.1 Why This Category Is “Free”

The category  $\mathcal{H}(\Omega)$  introduces no relations between histories. Each distinct sequence of commitments corresponds to a distinct morphism. This is the categorical analogue of the free monoid: the object generated by a set of generators with no imposed equations.

Traced monoidal categories, by contrast, construct quotients. The trace operator identifies morphisms that exhibit the same external behavior regardless of their internal paths. Spherepop constructs a free object: it preserves every detail of the causal history without collapsing any two distinct paths into a single morphism.

## 8.2 Paths Rather Than Loops

A geometrically useful way to describe the contrast is this. In traced monoidal categories the key structure is a loop. The feedback wire forms a cycle that is then collapsed. In Spheredpop the key structure is a directed path. Each commitment extends the path by one step. The identity of the computation is the path itself, and no step is collapsed or forgotten.

**Remark 8.1** (Path objects and causal categories). *The morphisms of  $\mathcal{H}(\Omega)$  are path objects in the sense of abstract homotopy theory: they are the paths themselves, not equivalence classes of paths. Their identity is literal equality of ordered sequences. This is also why Spheredpop computations feel structurally similar to event-sourcing logs and git commit graphs—the mathematics is the same idea expressed at different scales.*

## 9 Apparent Cycles from Dense Irreversible Chains

One counterintuitive feature of Spheredpop is that cyclic behavior can appear even though the underlying structure contains no loops. The option space shrinks monotonically and the history grows monotonically; the system never literally returns to a previous state. Yet an external observer may still perceive periodicity. This section explains how.

### 9.1 The Heartbeat Analogy

Consider a heartbeat. To an observer it appears as a cycle: systole, diastole, systole, diastole, repeating indefinitely. Yet time does not reset after each beat. The underlying causal history of the heart grows continuously. Each contraction is a distinct event in a strictly increasing sequence.

The cycle is a macroscopic pattern in a growing history—not a loop in the underlying causal structure.

### 9.2 Formal Statement

Suppose a Spheredpop computation operates over a sequence of periodically refreshed option spaces. At each refresh stage the computation again has access to a similar set of possibilities. If the commitment rule selects the same element at each stage, the observable output sequence via COLLAPSE will appear periodic.

**Proposition 9.1** (Apparent cycles from dense irreversible chains). *Let  $\{x_k\}$  be a sequence of commitments produced by a Spheredpop computation over refreshed option spaces, and suppose*

$x_{k+p} = x_k$  for some period  $p$  and all  $k \geq k_0$ . Then the output sequence produced by Collapse is periodic with period  $p$ , while the history  $H = [x_1, x_2, \dots]$  is a strictly non-repeating list.

*Proof.* Periodicity of the output follows because the transformation sequence repeats with period  $p$  in its suffix, so the composite transformation repeats accordingly. Non-repetition of the history follows because the history is an append-only list:  $H_{t+1}$  strictly contains  $H_t$  as a proper prefix, so no two prefixes of different length are equal.  $\square$

### 9.3 Practical Implications

Spherepop can model cyclic phenomena—heartbeats, circadian rhythms, oscillating chemical reactions, periodic regulatory feedback—without introducing any cyclic structure into its internal representation. The cycles are macroscopic patterns in a monotonically growing history.

This is precisely the structure of real physical and biological systems: the universe does not rewind after each orbital period or each heartbeat. The underlying causal history grows continuously while macroscopic patterns repeat.

## 10 Connections to Append-Only Systems

The mathematical structure identified in the preceding sections—an append-only history, a shrinking option space, and an observable state derived by composing committed transformations—appears under different names in several domains. These are not analogies imposed from outside; they are instances of the same algebraic structure.

**Event sourcing.** In event-sourced architectures the state of a system is not stored directly but derived from an append-only log of events. Each event is immutable once recorded, and the current state is reconstructed by replaying the event log. This is precisely the COLLAPSE operator: the observable state is  $T_{x_n} \circ \dots \circ T_{x_1}(s_0)$ , where each  $x_i$  is an event and  $s_0$  is the initial aggregate state.

**Version control.** A git repository is an append-only directed acyclic graph of commits. Each commit records a transformation of the repository state relative to its parents. The commit log is a commitment history; POP corresponds to committing; BIND corresponds to pre-commit hooks or merge filters that restrict admissible changes; and COLLAPSE corresponds to checking out a branch, reconstructing the working tree by replaying its history.

**Ledgers and blockchains.** A financial ledger records transactions sequentially. Account balances are reconstructed by replaying the ledger. A blockchain makes the append-only property cryptographically enforced: each block references the hash of its predecessor, making retrospective alteration computationally infeasible. Both structures are instances of the free history category  $\mathcal{H}(\Omega)$ .

**Causal sets in physics.** In the causal set approach to quantum gravity, spacetime is modeled as a locally finite partial order of events where the partial order encodes causal precedence. The history of a physical system is the causal set of events leading to its current state, and the state is derived by composing the effects of those events in causal order—formally identical to the COLLAPSE operator.

## 11 Spherepop Patterns in Programming Practice

Although Spherepop has been introduced as a formal framework, the underlying pattern is already familiar to programmers.

### 11.1 Scope Resolution as Spherepop

Modern programming languages implement lexical scoping through a hierarchy of nested environments. When the interpreter encounters a symbol, it searches through environments from innermost to outermost until a matching binding is found. This search can be represented as a sequence of nested domains

$$E_0 \subseteq E_1 \subseteq E_2 \subseteq \cdots \subseteq E_n,$$

where each environment extends the previous one.

Resolving an identifier involves progressively eliminating candidate environments. Each scope check acts as a BIND operation: it reduces the set of possible interpretations for the symbol. When the correct binding is identified, the selection corresponds to POP: the chosen interpretation becomes the committed meaning of the identifier. Evaluating the associated expression corresponds to COLLAPSE.

Scope resolution is therefore not merely a technical feature of programming languages but an instance of the broader Spherepop pattern: a search through nested domains that progressively restricts possibilities until a commitment is made.



## 11.2 Parser Combinators and Backtracking

Parser combinators that employ backtracking also exhibit the Spherepop structure in a modified form. A parser is presented with a string and must identify which grammar production matches. It progressively eliminates candidate productions via BIND-like filtering, eventually committing to a parse via a POP-like selection. The parse tree is the result of COLLAPSE applied to the commitment history.

The difference from pure Spherepop is that backtracking parsers can in principle reverse earlier commitments—but this possibility is visible in the structure: backtracking leaves a trace, and any implementation of it is itself a sequence of operations in the history.

## 12 Nested Domains and Problem Solving

The same structural pattern appears in human problem solving. When confronting a complex task, individuals rarely explore the entire space of possibilities simultaneously. Instead they work through a sequence of nested constraints.

At each stage certain options are ruled out by context, resources, or prior decisions. The remaining possibilities form a reduced option space from which the next action is selected. This iterative narrowing mirrors the Spherepop state structure.

The evolving situation is represented as the pair  $(\Omega_t, H_t)$ , where  $\Omega_t$  represents viably remaining options and  $H_t$  records the sequence of commitments already made. Each new decision reduces  $\Omega_t$  while extending  $H_t$ . The final outcome is the cumulative effect of the entire commitment history.

This interpretation suggests that Spherepop captures a general pattern by which decisions unfold in constrained environments—not merely a specialized computational formalism.

## 13 Reactive Pipelines in Practical Activity

Many practical activities follow a reactive pipeline structure that is naturally described in Spherepop terms.

**Painting.** A blank canvas initially admits an enormous range of possible images. Each brushstroke restricts what can be painted next. Once pigment has been applied the previous state cannot be restored without leaving evidence of alteration. The completed painting is the result of a long chain of irreversible commitments. The identity of the painting, in the

Spherepop sense, is not just its final appearance but the ordered sequence of strokes that produced it.

**Cooking.** Ingredients form an option space of potential dishes. Preparation steps such as chopping, heating, and seasoning progressively restrict the possibilities. Once ingredients are combined or cooked the previous configuration cannot be recovered. The finished dish is the COLLAPSE of the transformation history.

**Construction.** Early design choices restrict later possibilities. Structural decisions about foundations, materials, and layout eliminate entire branches of potential designs. The final building is not simply a static object but the cumulative result of a sequence of commitments made during construction, none of which can be undone without tearing down what followed.

**Childcare.** Caregivers continually respond to changing conditions by making small adjustments—feeding, redirecting attention, setting boundaries, altering routines. Each response changes the environment in which future actions occur. Over time these accumulated responses shape the developmental trajectory of the child. The causal history of those responses is not irrelevant detail; it is the explanation of how the present configuration came to be.

Across all these domains the underlying structure is the same: a system begins with a broad space of possibilities and evolves through a sequence of irreversible commitments. The final outcome emerges from the history of those commitments rather than from any single decisive step.

## 14 Spherepop as a General Decision Topology

The examples of the preceding sections suggest that Spherepop may be interpreted as a topology of decision processes. Rather than modeling computation purely as functional evaluation, the framework models it as the traversal of a narrowing landscape of possibilities.

Each step of the process transforms the decision topology in two ways. Contextual constraints reshape the available region. Commitments move the system along a specific path through that region. The resulting structure is a growing causal trajectory whose identity is determined by its ordered sequence of decisions.

This perspective helps explain why append-only histories appear so frequently in computing, engineering, and social systems. Whenever commitments are irreversible and context accumulates over time, the underlying dynamics resemble the Spherepop algebra. The framework is not an unusual construction imposed on well-behaved mathematical objects; it is the

natural algebra of processes that actually occur in the world.

## 15 Spherepop and Human Cognition

The Spherepop framework provides a natural language for describing aspects of human cognition. Reasoning processes frequently unfold as a progressive narrowing of possibilities rather than as a direct evaluation of a predetermined function.

When humans attempt to understand a situation they rarely compute the final answer in a single step. Instead they explore a space of candidate interpretations, gradually eliminating those inconsistent with available evidence or constraints. Each new piece of information acts as a BIND-like restriction that reduces the remaining option space. A decision or inference performs a POP, selecting one interpretation and recording it as part of the evolving reasoning history. The resulting mental model of the situation corresponds to COLLAPSE applied to the accumulated commitments.

The reasoning process cannot be reduced to its final conclusion. Two individuals may reach the same answer while following entirely different chains of inference. In standard logical formalisms these chains are collapsed once the conclusion is known. But in scientific explanations, legal arguments, and engineering designs the reasoning history matters: the conclusion acquires its justification from the ordered sequence of steps that produced it, not merely from its truth value.

Cognitive science increasingly recognizes that human reasoning is path dependent. Early assumptions constrain later interpretations, and the order in which information is encountered can influence the final outcome. Spherepop provides a concise mathematical description of this phenomenon: cognition unfolds as an append-only history of commitments within a dynamically shrinking space of possibilities.

Thought may thus be understood as the traversal of a possibility landscape through successive commitments that progressively stabilize an interpretation of the world. In systems governed by irreversible commitments, whether computational, physical, or cognitive, the identity of a process resides in its history.

## 16 Conclusion

The central theme of this work can now be stated precisely. Many important systems in computation, engineering, and cognition do not behave like pure functions whose identity is determined solely by their input-output relation. Instead they behave like histories: ordered

sequences of irreversible commitments whose cumulative effect produces the observable state of the system.

Traditional semantic frameworks, including traced monoidal categories, achieve their power by abstracting over internal computation. The trace operator collapses feedback loops and identifies morphisms that exhibit the same external behavior regardless of internal path. For many purposes this abstraction is both elegant and appropriate. But the systems examined in this paper belong to a different structural class. In these systems the internal path cannot be discarded without losing essential information: the provenance, order, and content of intermediate commitments are themselves part of the identity of the process.

Spherepop addresses this class of systems by preserving the commitment history rather than collapsing it. The state is represented as the pair  $(\Omega_t, H_t)$ , recording both what remains possible and what has already been committed. Three primitive operators govern evolution: BIND restricts the option space without making a commitment; POP performs an irreversible commitment and records it in the history; COLLAPSE derives the observable state by composing the transformations associated with each committed element in temporal order.

The resulting structure is the free history category  $\mathcal{H}(\Omega)$ , whose morphisms correspond to elements of the free monoid  $\Omega^*$ . No two distinct histories are identified. Where traced monoidal semantics constructs a quotient that collapses internal paths into external functions, Spherepop constructs a free object that preserves every causal detail.

This structure appears naturally across many domains: in event-sourced architectures that reconstruct state by replaying append-only logs; in version control systems that represent software evolution as commit histories; in financial ledgers and blockchains that enforce append-only transaction sequences; in scope resolution and parser combinators within programming languages; in the iterative narrowing of everyday problem solving and practical activity; in causal set models of physical spacetime; and in the path-dependent character of human reasoning.

Across all of these domains the same structural pattern recurs. A system begins with a space of possibilities, evolves through contextual restrictions and irreversible commitments, and produces observable states by composing the effects of those commitments in temporal order.

Spherepop provides a minimal algebraic language for describing this pattern. Where traditional semantics begins with states and abstracts away the path that produced them, Spherepop begins with the path and allows the state to emerge as its shadow. In systems governed by irreversible commitments, history is not an auxiliary record of what happened but the structure that defines what the system is. When the past cannot be undone, the

sequence of events that produced the present becomes inseparable from the present itself.